

- **A* è ottimamente efficiente per qualsiasi euristica:**
non esiste alcun altro algoritmo ottimo che garantisca di espandere meno nodi di quelli espansi da A*
- Purtroppo il numero di nodi espansi aumenta esponenzialmente con la profondità della soluzione ottima
- A* mantiene in memoria tutti i nodi generati (è una ricerca in ampiezza)

Ridurre i requisiti di memoria di A*

- **IDA***: unisce A* e iterative deepening (non lo studiamo)
- **RBFS**: recursive best-first search:
 - Simile alla *ricerca ricorsiva in profondità* con una differenza importante: usa un “**upper bound**” dinamico che consente di focalizzare la ricerca **sul percorso più promettente** invece di continuare indefinitamente lungo lo stesso percorso.
 - Questo upper bound (limite superiore) ricorda la migliore alternativa fra i percorsi attualmente aperti

Recursive Best-First Strategy (RBFS)

- **Difetto:**
lo stesso nodo può essere visitato più volte, se l'algoritmo, dopo aver cambiato percorso, ritorna al percorso precedentemente abbandonato
- **Pregio:**
poche esigenze di spazio, come la *ricerca in profondità senza backtracking*: mantiene solo i nodi del percorso di ricerca corrente e i loro fratelli. È un vantaggio rispetto ad A* che mantiene informazione su tutti i nodi aperti e chiusi

Recursive best first strategy (RBFS)

- RBFS utilizza:
 - Un nodo
 - Un limite superiore (**upper bound**) locale del nodo
- Esplora:
 - il sottoalbero del nodo finché nella sua frontiera ci sono nodi i cui costi non eccedono il limite superiore
- Upper bound di un nodo:
 - $\min(\text{upper_bound_parent}, \text{valore_fratello_di_costo_minimo})$
 - memorizza la stima del costo del percorso alternativo migliore,

Dall'articolo di Korf 1993

- L'algoritmo ha 3 argomenti:
 - Un nodo **N**,
 - Un valore **F(N)** ad esso associato,
 - un upper bound **B**
- **f(n)**: la **funzione di valutazione**, è statica cioè il valore restituito non cambia nel tempo
- **F(n)**: valore associato al nodo n, è dinamico cioè cambia nel tempo e dipende dai discendenti di N:
 - $F(N) = f(N)$ se N è esplorato per la prima volta
 - $\min(F(S_N))$ con S_N sottoalbero di N altrimenti

Linear-space best-first search, Richard E. Korf, Artificial Intelligence 62 (1993) 41-78 Elsevier

Dall'articolo di Korf 1993

- L'algoritmo ha 3 argomenti:
 - Un nodo **N**,
 - Un valore **F(N)** ad esso associato,
 - un upper bound **B**
- **f(n)**: la **funzione di valutazione**, è statica cioè il valore restituito non cambia nel tempo
- **F(n)**: valore associato al nodo n, è dinamico
- **B**: è calcolato basandosi sui valori F(.) dei nodi fratelli, ricorda il secondo migliore

Linear-space best-first search, Richard E. Korf, Artificial Intelligence 62 (1993) 41-78 Elsevier

Dall'articolo di Korf 1993

- L'algoritmo ha 3 argomenti:
 - Un nodo **N**,
 - Un valore **F(N)** ad esso associato,
 - un upper bound **B**
- **La chiamata iniziale** a RBFS è:

RBFS (r, f(r), ∞)

dove r è il nodo radice r, il valore F(r) è pari a quello statico f(r) e l'upper bound è pari a ∞

- **Esempio: RBFS(Arad, 366, ∞)**

Linear-space best-first search, Richard E. Korf, Artificial Intelligence 62 (1993) 41-78 Elsevier

- RBFS lavora come A* fintantoché la soluzione che costruisce rispetta l'upper bound (è la migliore)
- Sospende la ricerca lungo un cammino quando questo non appare più il migliore
- Il cammino viene dimenticato (nodi cancellati)
- Viene solo conservata traccia nella sua radice del costo che si era stimato esplorando quella via

Algoritmo RBFS (Korf 1993)

RBFS (node: N , value: $F(N)$, bound: B)

IF $f(N) > B$, return $f(N)$ // la fz. val. supera il limite superiore, cambia percorso

IF N is a goal, **EXIT** algorithm // **successo!**

IF N has no children, **RETURN** infinity // vicolo cieco, cambia percorso

FOR each child N_i of N , // **inizializza** $F(.)$ per i successori di N

```
IF f(N)<F(N), F[i] := MAX(F(N),f(Ni)) // N è il nodo padre, su cui siamo focalizzati
```

ELSE $F[i] := f(Ni)$

sort Ni and F[i] in increasing order of F[i] // ordinamento per F crescenti, dopo

```
// F[1] sarà l'F del figlio più promettente
```

IF only one child, $F[2] := \text{infinity}$

```
WHILE (F[1] <= B and F[1] < infinity) // discesa ricorsiva solo se upper bound rispettato
```

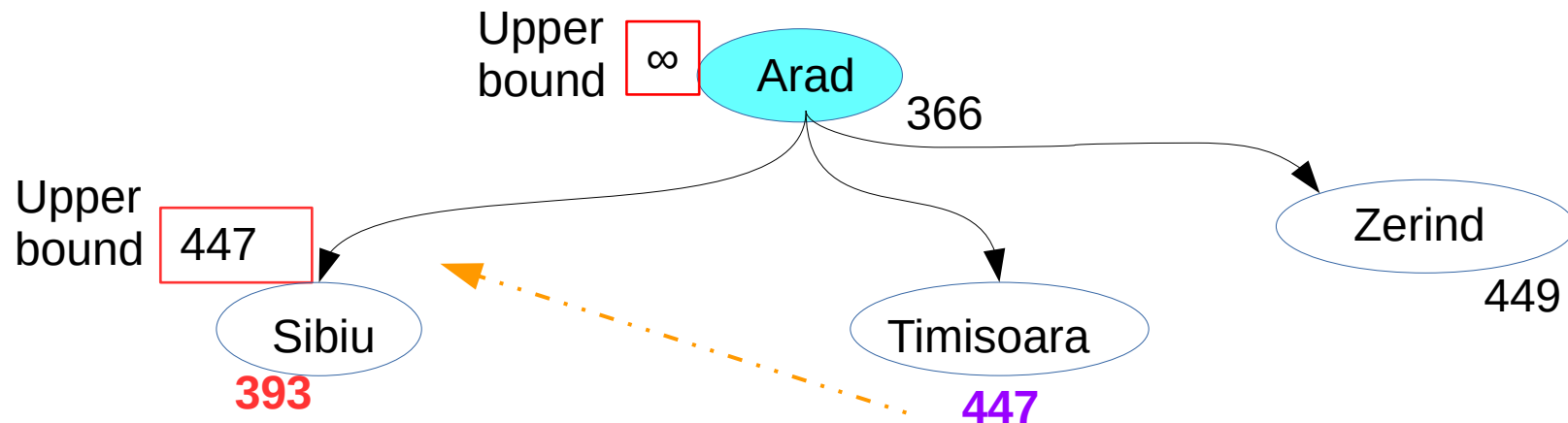
```
F[1] := RBFS(N1, F[1], MIN(B, F[2])) // applico ricorsivamente RBFS
```

insert N1 and F[1] in sorted order

```
return F[1]
```

NOTA: $F[i]$ è il valore F del nodo in posizione i dopo l'ordinamento. La posizione 1 è quella del nodo più promettente, la 2 quella dell'alternativa migliore e così via.

Esempio 1/5



Viene scelta Sibiu perché è il nodo sul percorso ritenuto più conveniente Viene associato a tale nodo il valore 447, che è la stima di costo della sua alternativa più conveniente

$F[\text{Arad}] = f(\text{Arad}) = 366$
 $\text{Upper bound}[\text{Arad}] = \infty$

For each child N_i of N :

$f(N) < F(N)?$

$366 < 366?$ No quindi $F[i] := f(N_i)$

esempio $F[\text{Sibiu}] := f(\text{Sibiu})$ cioè 393

Sort basato su F :

Sibiu (393), Timisoara (447), Zerind (449),
quindi $F[1]$ è $F[\text{Sibiu}]$

CICLO WHILE:

$F[\text{Sibiu}] < \text{Upper bound}[\text{Arad}]$? Sì!

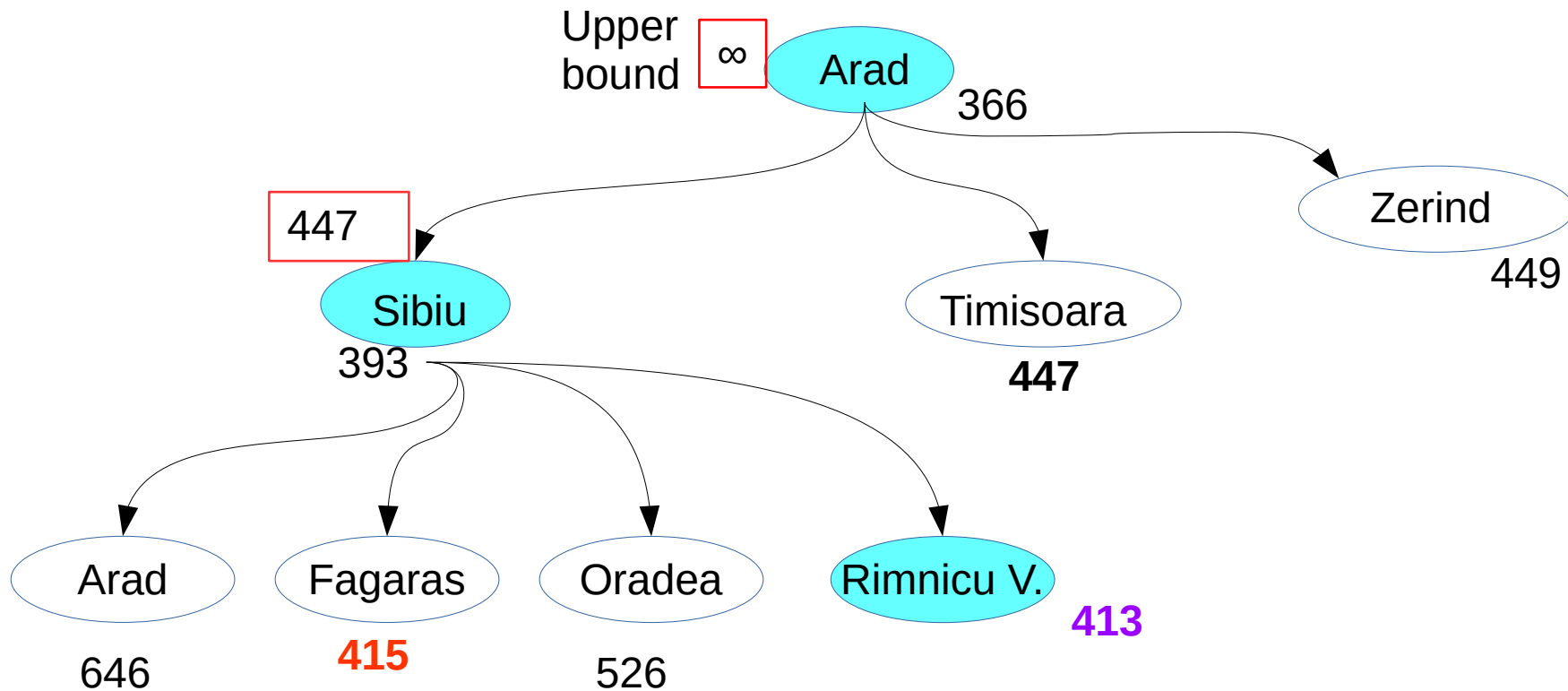
$F[\text{Sibiu}] < \text{infinity}$? Sì!

$F[1] := \text{RBFS}(N1, F[1], \text{MIN}(B, F[2]))$

Dove $B = \infty$, 2 corrisponde a Timisoara e $F[2] = 447$:

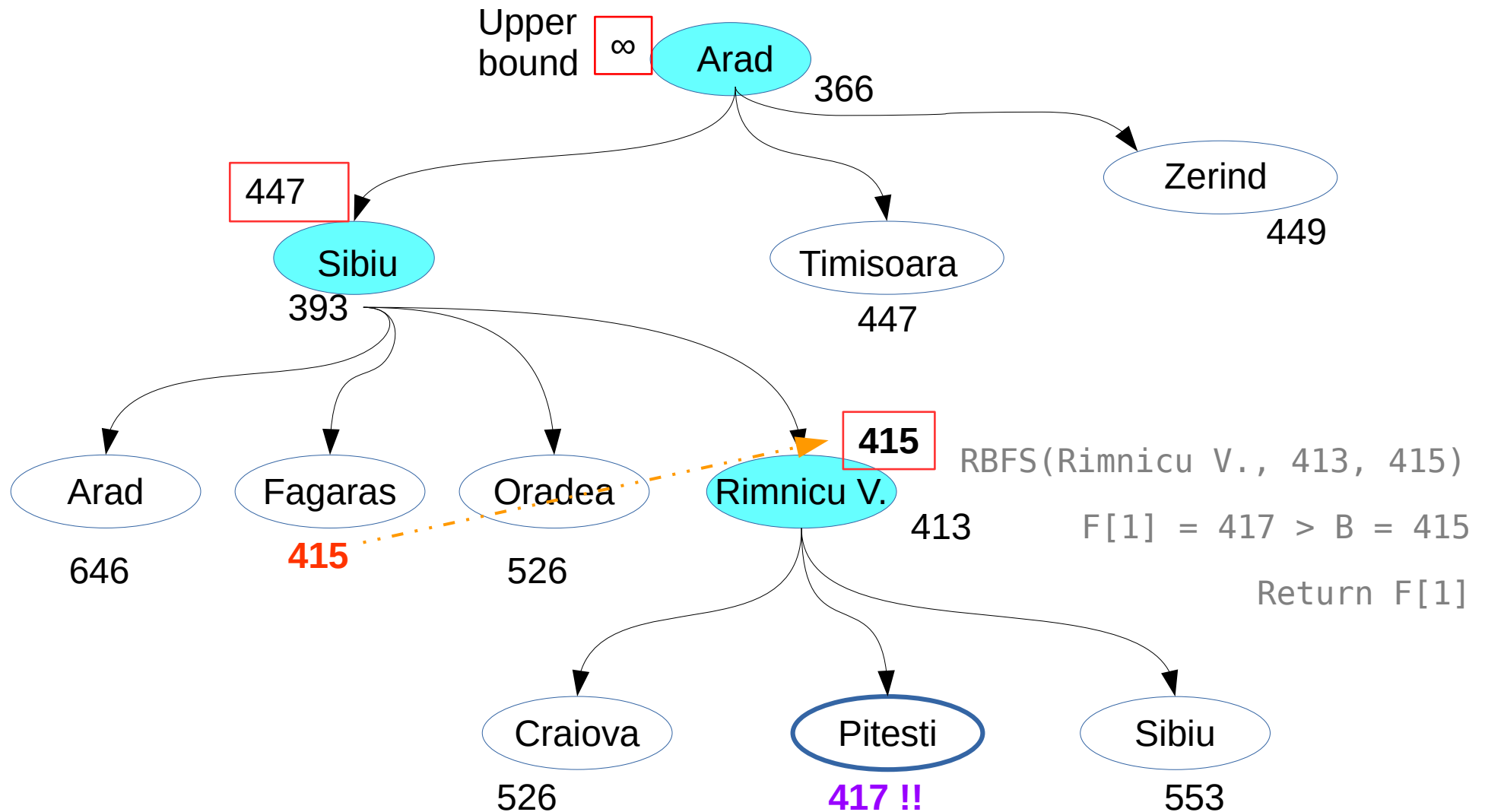
$F[1] := \text{RBFS}(\text{Sibiu}, 393, 447)$

Esempio 2/5



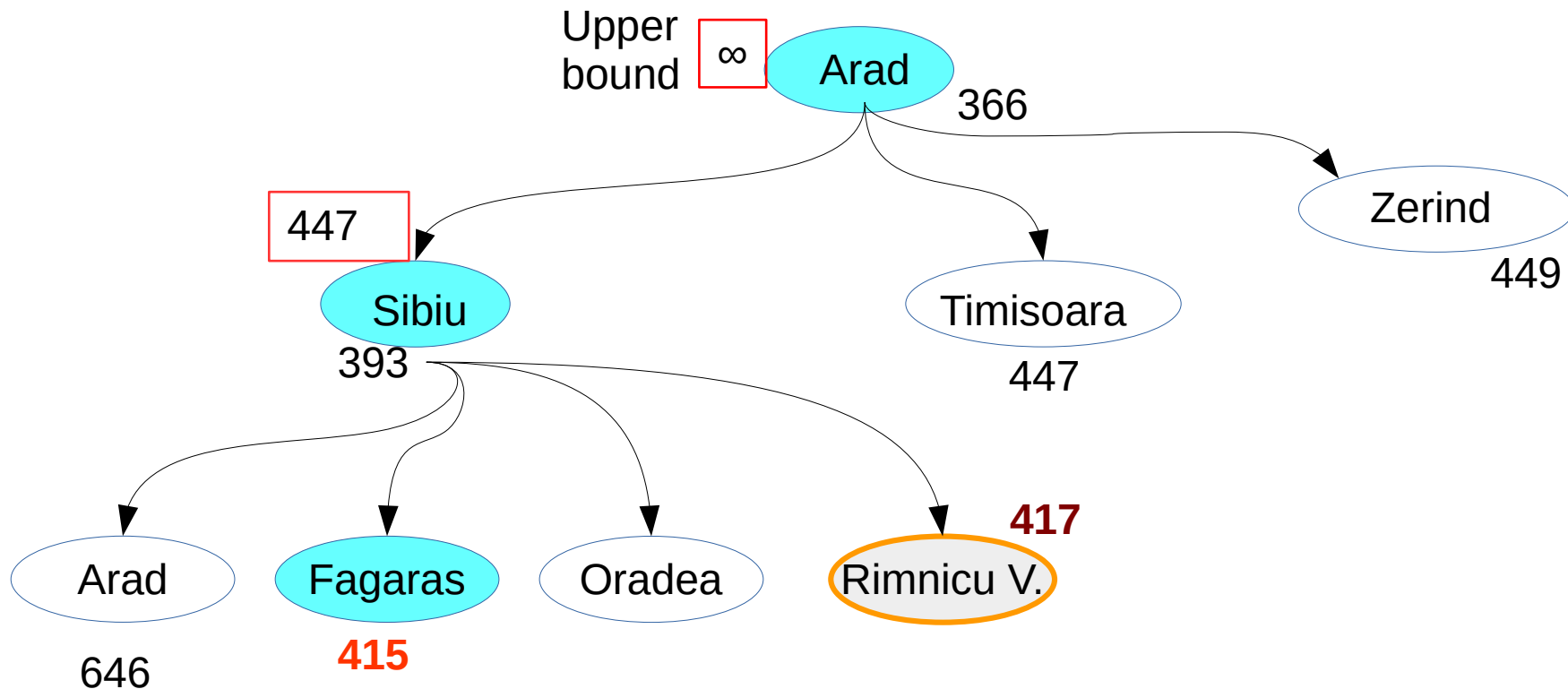
Rimnicu Vilcea è il nodo più promettente.
Fagaras è il secondo migliore, quindi 415 sarà l'upper bound per il richiamo ricorsivo su Rimnicu V.

Esempio 2/5



Pitesti ha una stima di costo più elevata dell'alternativa più conveniente a Rimnicu Vilcea, la condizione del while fallisce: si cambia percorso

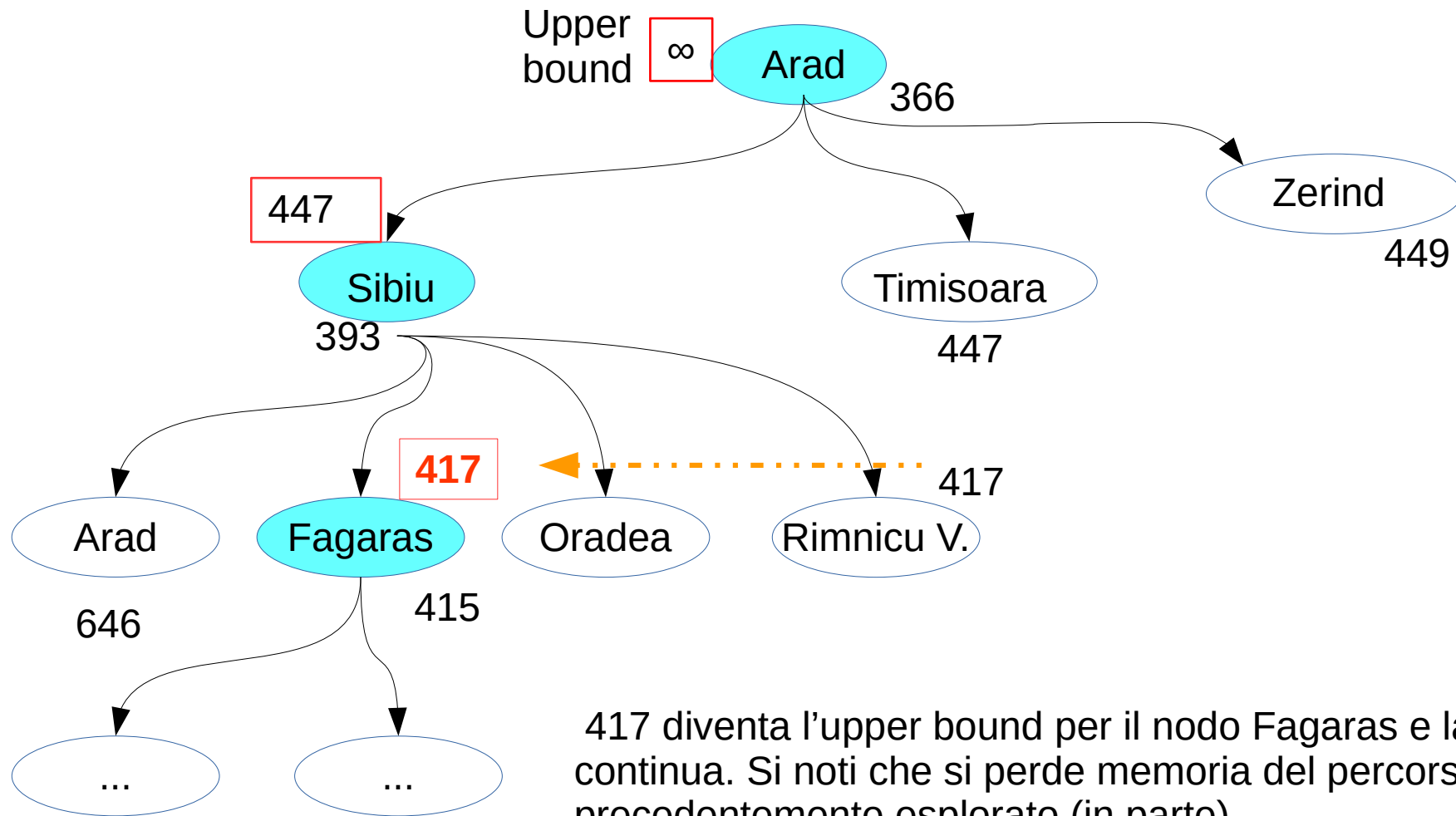
Esempio 3/5



La valutazione di Rimnicu Vilcea viene aggiornato alla valutazione del suo discendente più promettente (Pitesti). I nodi figli di Sibiu sono riordinati di conseguenza. Ora il nodo più promettente è Fagaras

Il sottoalbero di Rimnicu Vilcea è stato rimosso

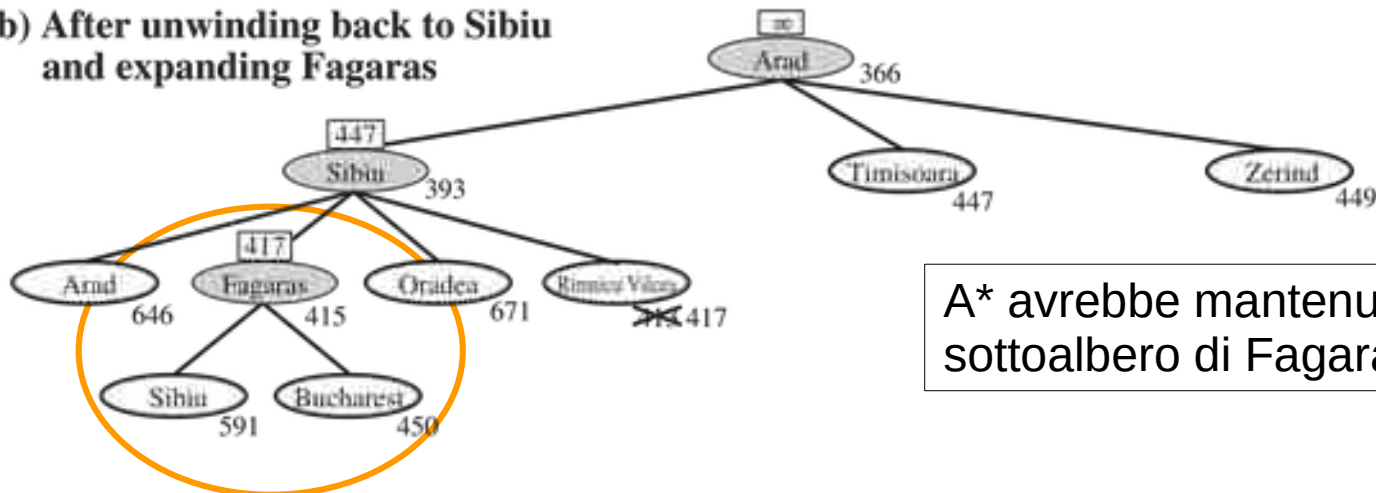
Esempio 4/5



417 diventa l'upper bound per il nodo Fagaras e la ricerca continua. Si noti che si perde memoria del percorso precedentemente esplorato (in parte)

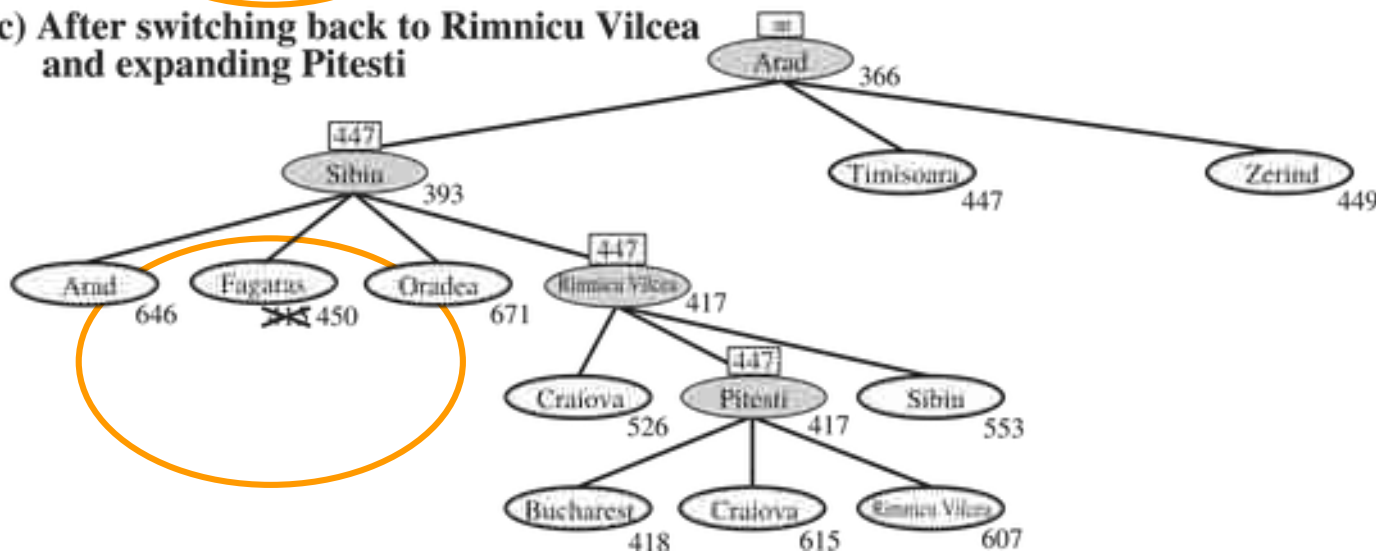
Esempio 5/5

(b) After unwinding back to Sibiu and expanding Fagaras



A* avrebbe mantenuto anche i nodi del sottoalbero di Fagaras

(c) After switching back to Rimnicu Vilcea and expanding Pitesti



- RBFS è **ottimo** se l'euristica è **ammissibile**
- **Complessità spaziale lineare:** $O(bd)$
- **Complessità temporale:** difficile da definire perché dipende dall'accuratezza dell'euristica
- Non si accorge di cammini ripetuti
- Sfrutta poco la memoria: Non riesce a sfruttare aumenti nella disponibilità della memoria per incrementare l'efficienza